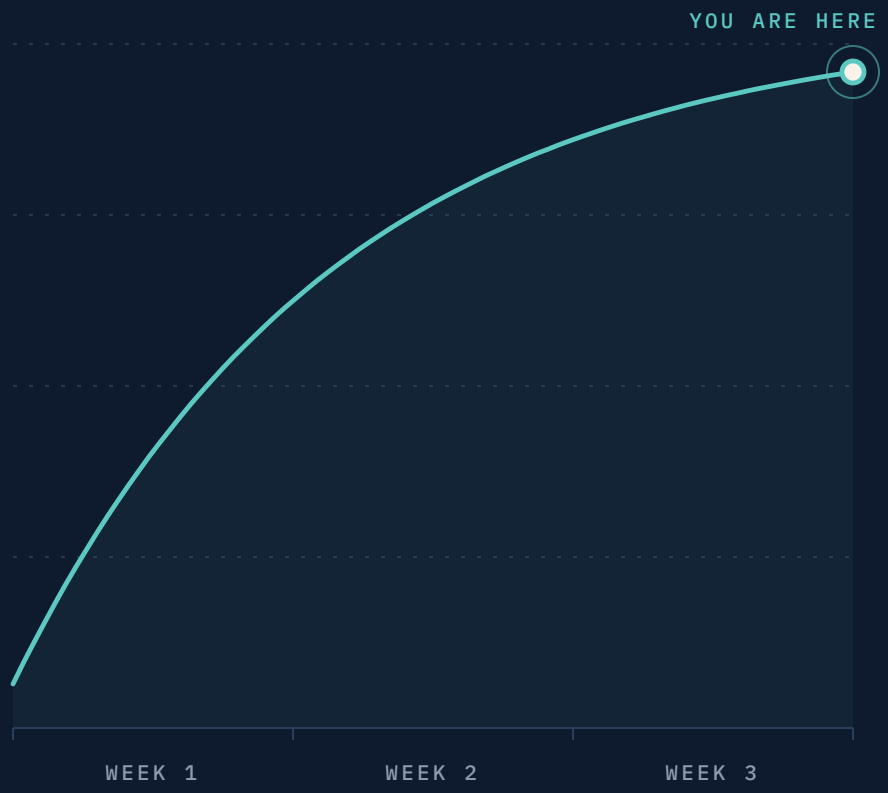
WEEK 3 · DAY 5 · OVERVIEW

Course 4 Capstone



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 3 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

A colleague who covers emerging market corporates hands you a public file: financial statements of Polish companies, 64 ratios each, and a flag for the companies that went bankrupt within a year.

The dataset page says the bankrupt companies were analysed from 2000 to 2012 and the ones still operating from 2007 to 2013 (dataset page).

It is one country in one period, and that market differs from US credit.

She asks which of three standard models best separates the companies that went bankrupt, how well on companies it never saw, and where it fails.

A model's output here is "the model's probability of bankruptcy in this file", or "flagged at threshold t", never a credit view on any company.

The work is yours: each question has an empty cell and a **check()** line, and the solutions notebook is one worked answer.

OBJECTIVES

Describe the file's gaps, copies and extremes, and look at it without the label.

Fit a logistic regression and two pruned trees inside pipelines, and choose every setting and a threshold for a recall of 0.70 on folds of the training rows.

Open the test rows once, then describe what the chosen model leans on and where it fails.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 3 EXERCISES

Your Route Through Day 5

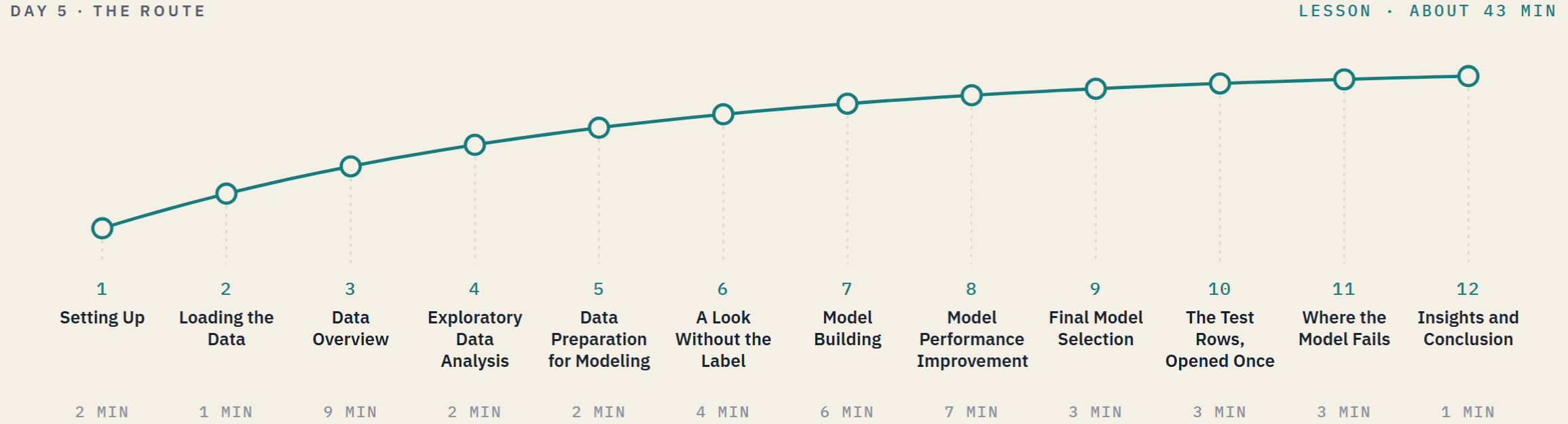


FIGURE 2 Twelve sections, in order, with about how long each takes.

Time. About 43 minutes for the lesson, then about 9 for the three exercises.

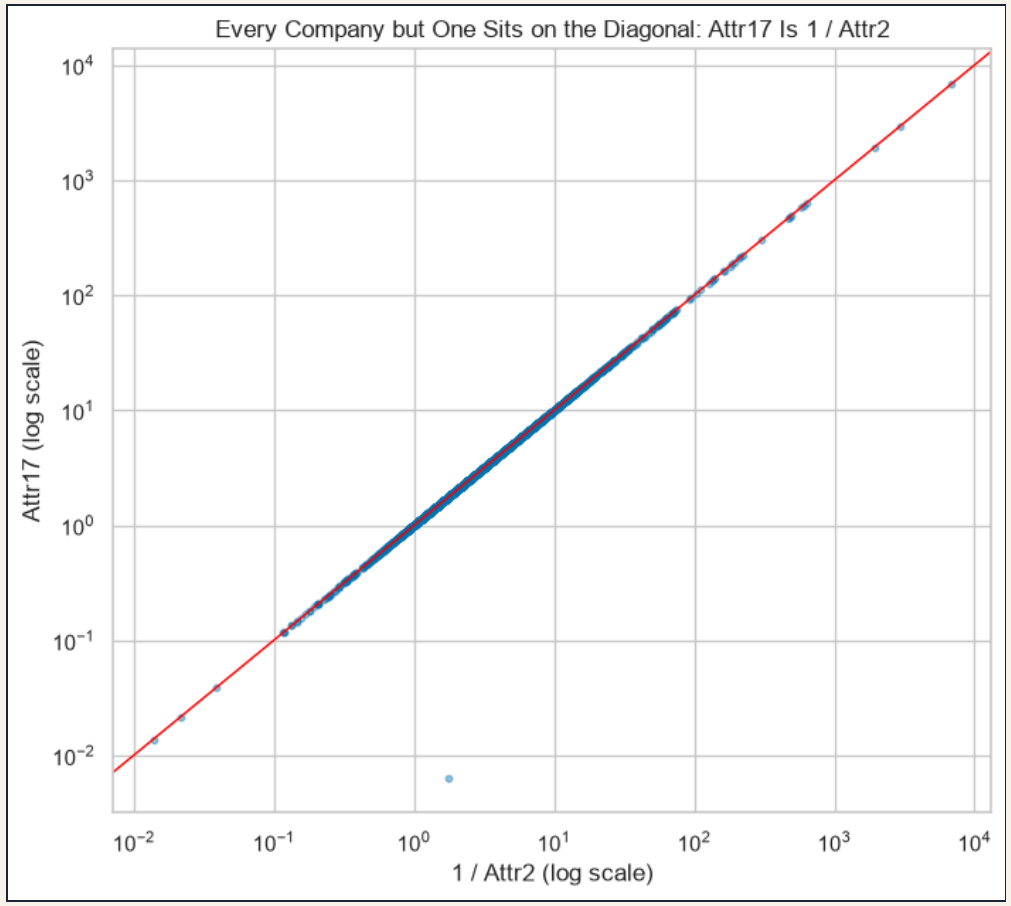
Setting Up

THE ANSWER CHECKER

```
# check() compares a fingerprint of your answer with a stored one, so no answer is written on the page
import hashlib
def check(label, answer, code, places=None):
    if answer is ...:
        print(label + ": not attempted yet")
        return
    if places is not None:
        answer = round(float(answer), places)
    text = label + "|" + str(answer).strip().lower()
    # hashlib.sha256() makes a fingerprint that cannot be turned back into the answer;
    # .encode() turns the text into bytes for it, and .hexdigest() writes it as letters and digits
    if hashlib.sha256(text.encode()).hexdigest()[:12] == code:
        print(label + ": correct")
    else:
        print(label + ": not yet. You have " + str(answer))
```

`check()` **compares a fingerprint of each answer with a stored one, so no answer is written on the page.**

Loading the Data



READ THE FILE

```
bankrupt = pd.read_csv(DATA + "polish_bankruptcy_5year.csv")
print(len(FEATURES), "features")
```

```
64 features
```

THE TEST OF THE FILE'S NAMES

The page's X17 is the reciprocal of its X2, so if the file's names follow the page, **Attr17** x **Attr2** is 1.

```
both = bankrupt[["Attr2", "Attr17"]].dropna()
within = ((both["Attr17"] * both["Attr2"]) - 1).abs() < 0.01
```

5,890 of 5,891 rows sit on the diagonal, so `Attrk` is read as the page's Xk from here on.

Count the rows, gaps, copies and extremes.

Data Overview

QUESTION	WHAT IT ASKS
Shape, Outcome and Dates	How many rows does the file hold, how many companies went bankrupt, and what share is that in percent?
Missing Values	How many cells are empty, how many columns and how many rows have at least one gap, and which column has the most?
Predict 1	Among the companies where **`Attr37`** is empty, what share went bankrupt, in percent?
Does a Gap Say Something?	Compute both shares in percent, each with Course 1's rough margin, 1.96 x sqrt(p x (1 - p) / n), and keep them in a table **`by_gap`**.
	Draw the two shares as dots, each with its margin as a whisker, and the whole file's share as a dashed line.
Copies	How many rows repeat an earlier row exactly, how many of the rows with a twin are bankrupt, and how many rows repeat the 64 features but not the outcome?
Extremes	Which feature holds the largest value in the file, and what is it?

Compare one margin ratio across the outcomes.

Exploratory Data Analysis

QUESTION	WHAT IT ASKS
Profit on Sales by Outcome	What is the median **`Attr39`** (X39, profit on sales / sales) of the companies that went bankrupt, and of those that did not?
	Draw a raincloud of **`Attr39`** for each outcome, for the companies between its 1st and 99th percentiles, the dots shifted a little with **`np.random.default_rng(SEED)`**.

THE FEATURES THE NOTEBOOK NAMES

- **`Attr2`** (X2): total liabilities / total assets
- **`Attr17`** (X17): total assets / total liabilities
- **`Attr21`** (X21): sales (n) / sales (n-1)
- **`Attr27`** (X27): profit on operating activities / financial expenses
- **`Attr32`** (X32): (current liabilities x 365) / cost of products sold
- **`Attr33`** (X33): operating expenses / short-term liabilities
- **`Attr37`** (X37): (current assets - inventories) / long-term liabilities
- **`Attr39`** (X39): profit on sales / sales
- **`Attr60`** (X60): sales / inventory
- **`Attr63`** (X63): sales / short-term liabilities

A RAINCLOUD PLOT

A **raincloud plot** stacks three views of one distribution: a half violin (the cloud), a narrow box under it, and every row as a dot (the rain).

DRAWING THE HALF VIOLIN

`sns.violinplot(split=True)` with no **`hue`** draws half a violin, and **`native_scale=True`** places it at the outcome's own number, 0 or 1.

Data Preparation for Modeling

QUESTION	WHAT IT ASKS
Copies Out, Then One Split	Drop the exact copies, then split the rest 80/20 into `X_train, X_test, y_train, y_test` with `stratify=y` and the course seed.

WHAT THE FEATURES ARE

Most features are ratios; X29 is a logarithm of total assets, X55 is working capital (an amount), and nine, X32 among them, are in days.

WHICH FEATURE NEVER HAS A GAP

The provider kept only statements with `Attr20` present, a filter named in the header of the fifth-year file in the UCI download, so `Attr20` never has a gap.

The preparation needs three things: the gaps kept as columns, the copies dropped, and Day 10's quantile transform for the extremes.

A Look Without the Label

QUESTION	WHAT IT ASKS
Many Features, Few Directions	Fit a pipeline of that imputer, Day 10's quantile transform and Day 14's **`PCA(random_state=SEED)`** on **`X_train`** (steps **`"impute"`**, **`"quantile"`**, **`"pca"`**), keeping its output as **`train_scores`**.
	Draw Day 14's scree plot of the first 20 components: each one's share as a bar, the running total as a line, and 80 percent dashed.
Four Clusters on Two Components	Run Day 12's **`KMeans(n_clusters=4, n_init=10, random_state=SEED)`** on the first two columns of **`train_scores`**, numbering the clusters by size, largest 0, as Day 12 does.
	Draw the training companies on the two components with each cluster's area shaded (Day 12's nearest-centre tiles: **`contourf`** over a grid, each point given its cluster by **`kmeans.predict`**), the bankrupt companies dark, and each area's size and bankrupt share in the key.

WHAT THESE TWO NEVER SEE

PCA and K-means never see the outcome.

FILLING THE GAPS

`SimpleImputer(strategy="median")` fills each gap with its column's median, learned at **`fit`** from the rows it is given.

Model Building

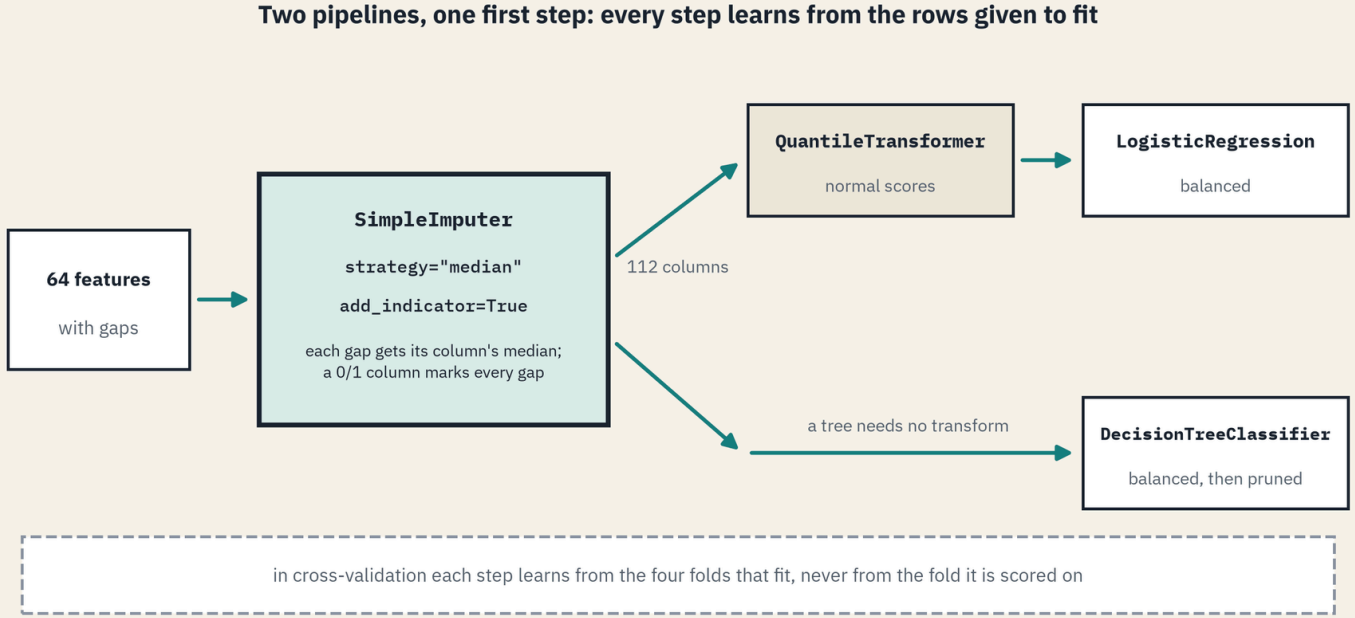


FIGURE 4 One median imputer with missing indicators, then a quantile transform and a logistic regression, or a tree.

THE CRITERION

With so rare a label ROC AUC can stay high while the top of the ranking is crowded with survivors, so models are ranked by **average precision** (Day 8) on five folds of the training rows.

WHAT THE SECTION ASKS

WHAT IT ASKS
Build **`logistic`**, a pipeline of that imputer, Day 10's quantile transform and **`LogisticRegression(solver="newton-cholesky", tol=1e-8, class_weight="balanced")`** (steps **`"impute"`**, **`"quantile"`**, **`"logit"`**).
Score the same pipeline with **`add_indicator=False`** on the same folds.
Build **`tree`**, a pipeline of the same imputer and **`DecisionTreeClassifier(class_weight="balanced", random_state=SEED)`** (steps **`"impute"`**, **`"tree"`**).

Model Performance Improvement

QUESTION	WHAT IT ASKS
Pre-Pruning	Search **`max_depth`** 4, 6, 8, 10 and **`min_samples_leaf`** 10, 20, 40 for **`tree`** with Day 9's **`GridSearchCV`** on **`folds`**, scoring **`"average_precision"`**, as **`pre_search`**.
	Draw the twelve fold scores as an annotated heatmap, depth down and leaf size across, with the best cell outlined.
Post-Pruning	Search **`ccp_alpha`** 0.0005, 0.001, 0.002, 0.003, 0.005, 0.01 for **`tree`** the same way, as **`post_search`**, and keep the best pipeline as **`post_pruned`**.
Thresholds for a Recall of 0.70	Put the three models in a dictionary **`models`** under the keys **`"logistic"`**, **`"pre-pruned tree"`** and **`"post-pruned tree"`**.
	Draw the three out-of-fold precision-recall curves on one chart, recall 0.70 dashed and each model's threshold as a dot.

HOW A PIPELINE'S SETTING IS NAMED

In a grid for a pipeline, a step's setting is named with the step, two underscores and the setting: **`tree__max_depth`**.

WHERE A THRESHOLD COMES FROM

Here the folds do that job: **`cross_val_predict(model, X_train, y_train, cv=folds, method="predict_proba")`** fits the model five times and gives each training company the probability from the fit that did not see it, an **out-of-fold** probability.

Choose one model by the fixed criterion.

Final Model Selection

QUESTION	WHAT IT ASKS
Judged on Their Own Rows	Fit **`logistic`** on the training rows (the searches already refitted the trees), and score all three there with the scoring function, each at its threshold.
The Choice	Which model goes forward, by the criterion set before any model was fitted?

THE SCORING FUNCTION

```
def model_performance_classification(model, predictors, target, threshold=0.5):
    flags = (model.predict_proba(predictors)[:, 1] >= threshold).astype(int)
    return pd.DataFrame({
        "Average precision": [average_precision_score(target, model.predict_proba(predictors)[:, 1])],
        "ROC AUC": [roc_auc_score(target, model.predict_proba(predictors)[:, 1])],
        "Accuracy": [accuracy_score(target, flags)],
        "Recall": [recall_score(target, flags, zero_division=0)],
        "Precision": [precision_score(target, flags, zero_division=0)],
        "F1": [f1_score(target, flags, zero_division=0)],
    })
```

Score the three models on unseen companies once.

The Test Rows, Opened Once

QUESTION	WHAT IT ASKS
The Test Table	Score the three models on the test rows in one table, **`test_comp`**, each at its threshold, and keep each model's test flags in a dictionary **`test_flags`**.
	Draw the three test confusion matrices side by side in Day 7's four colours.
	Draw a bump chart of the models' ranks on training-row and fold average precision and on the test table's average precision, ROC AUC, recall and precision.

A BUMP CHART

A **bump chart** ranks the models on each measure, first at the top, and joins each model's ranks with a line.

AN UPSET CHART

An **UpSet chart** counts overlapping groups: one bar per pattern, and under each bar a dot for every model in the pattern.

Where the Model Fails

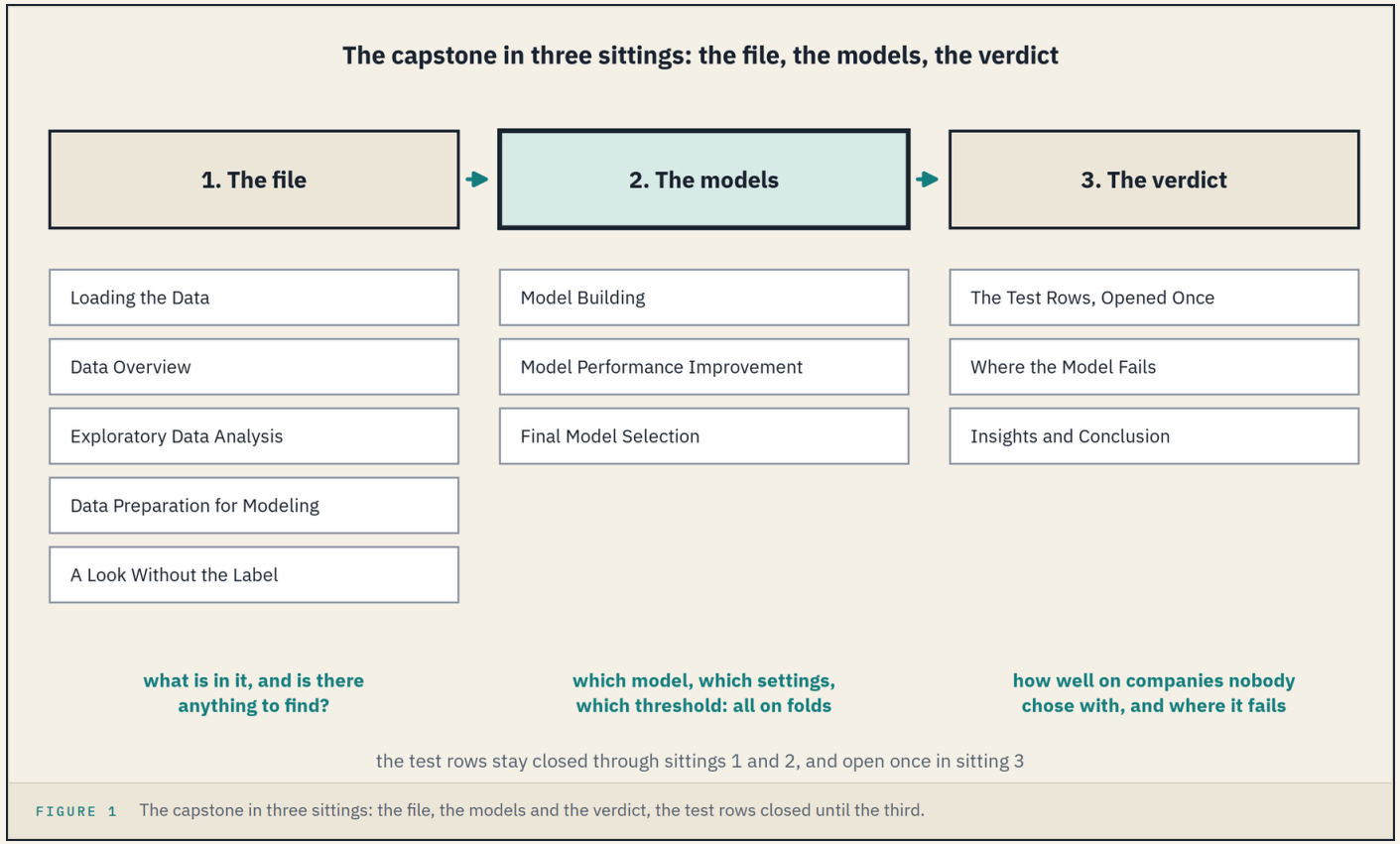


FIGURE 1 The capstone in three sittings: the file, the models and the verdict, the test rows closed until the third.

WHAT THE SECTION ASKS

WHAT IT ASKS
Name the 112 columns with `logistic["impute"].get_feature_names_out()`.
Draw the two top tens as lollipops side by side, the coefficients coloured by sign.
Which test bankruptcies does the logistic miss at its threshold?

Write the model's description in plain words.

Insights and Conclusion

QUESTION	WHAT IT ASKS
The Description	Write **`description`**, one string of a few sentences: what the chosen model separates and how well on the test rows, where it is wrong, what the data cannot support, and what was not tested.

THE REMINDER THE NOTEBOOK RUNS

```
found = [word for word in WATCH if word in str(description).lower()]
print("Words from the list in the description:", found)
```

```
Words from the list in the description: []
```

A model describes the file it was fitted on.

You Can Now

- Run a whole classification job on a real file, every choice made on training rows.
- Fill gaps inside a pipeline with **`SimpleImputer`** and its missing indicators.
- Read a file without its label with PCA and K-means.
- Choose settings with **`GridSearchCV`** on a pipeline, and a threshold from out-of-fold probabilities.
- Say what a model leans on and where it fails.

NEXT
WEEK 1 · DAY 1

Tree Instability and the Bootstrap

Course 5 grows forests and boosting from one tree; its week 3 returns to these rows.